# bita v0.13.0

Differential file synchronization over http(s).

#delta-update #differential #file-synchronization #sofware-update

**Follow**




## bita

*bita* is a HTTP based file synchronization tool striving for low bandwidth usage through data reuse.

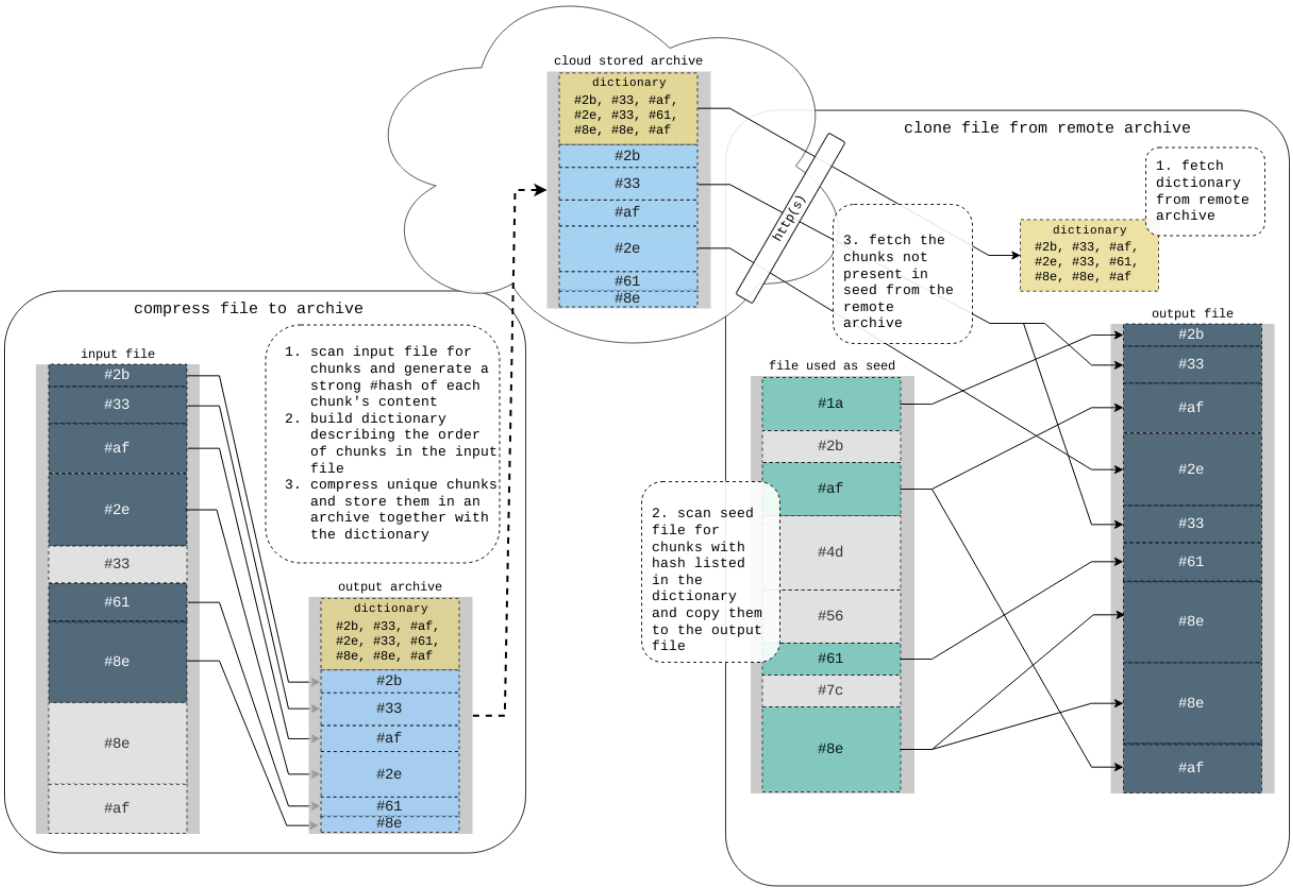
- Clone from remote while reusing data from any local file or device 📁
- Clone using a file or block device as output 💾
- Host archives using any regular HTTP/HTTPS server or service 🔗
- Include in your own project with the bitar library 💫
- Written in Rust for fun, performance and robustness 🚀 🖤

## Software updates

*bita* is a generic file synchronization tool but has been developed with software update of embedded/IoT systems in mind.

Software update is a typical case where *bita* may provide significant bandwidth reductions, where one can expect that a new software image will contain a lot of data already present on the system being updated. *bita* can identify the parts (*chunks*) already present on the system and fetch the missing ones from remote, still outputing an exact clone of the archived source file.

No need to pre-build patch files for going to/from different release versions. No need to run any special file server. Just `bita compress` the release image, upload the archive to any HTTP file hosting site. And `bita clone` the archive using whatever local data is available on the system.




## Compressing

On compression the input file is scanned for chunk boundaries using a rolling hash. With the default setting a suitable boundary should be found every ~64 KiB. A chunk is defined as the data contained between two boundaries. For each chunk a strong hash is generated (using blake2). The chunk location (offset and size) in the input file and the strong hash is then stored in the dictionary. If chunk's strong hash has not been seen before the chunk data is also compressed (using brotli) and inserted into the output archive.

The final archive will contain a dictionary describing the order of chunks in the input file and the compressed chunks necessary to rebuild the input file. The archive will also contain the configuration used when scanning input for chunks.

## Cloning

On clone the dictionary and chunker configuration is first fetched from the remote archive. Then the given seed file(s) are scanned for chunks present in the dictionary. Scanning is done using the same configuration as when building the archive. Any chunk found in a seed file will be copied into the output file at the location(s) specified by the dictionary. When all seeds has been consumed the chunks still missing, if any, is fetched from the remote archive, decompressed and inserted into the output file.

To keep the HTTP overhead low while cloning all adjacent chunks are fetched with a single request. And if possible the same connection is used for the whole clone operation.

*bita* can also use the output file as seed and reorganize chunks in place. Except using this for the obvious reason of saving bandwidth this will also let *bita* avoid writing chunks that are already in place in the output file. This may be useful if writing to storage is either slow or we want to avoid tearing on the storage device.

Each chunk, both fetched from seed and from archive, is verified by its strong hash before written to the output. *bita* avoids using any extra storage space while cloning, the only file written to is the given output file.

## Scanning for chunks

The process of splitting a file into chunks is heavily inspired by the one used in rsync. Where a window of bytes (default 64 for RollSum and 20 for BuzHash) is sliding through the input data, one byte at a time.

For every position of the window a short checksum is generated. If we're assuming that the checksum has an even distribution we can say that with some probability this checksum will be within a range of values at every `n` interval of bytes, where `n` represents the average target chunk size.

When the checksum is within this range a chunk boundary has been found. A strong hash (blake2) is then generated for the data between the last boundary and this one. The strong hash is the one used to identify this chunk while the weaker rolling hash is never stored but only used for finding chunk boundaries.

The average target chunk size and the upper/lower limit of a chunk's size is runtime configurable.

## Server requirements

The server serving *bita* archives can be any HTTP/HTTPS server supporting range requests, which should be most.

## Install from crates.io

Install *bita* using cargo:

```
olle@home:~$ cargo install bita
```

## Build from source

```
olle@home:~$ cargo build
```

Build in release mode with rustls TLS backend:

```
olle@home:~$ cargo build --release --no-default-features --features rustls-tls
```

## Example usage

Create a compressed archive `release_v1.1.ext4.cba` from file `release_v1.1.ext4` :

```
olle@home:~$ bita compress -i release_v1.1.ext4 release_v1.1.ext4.cba
```

Clone using block device `/dev/mmcblk0p1` as seed and `/dev/mmcblk0p2` as target:

```
upgrader@device:~$ bita clone --seed /dev/mmcblk0p1 https://host/release_v1.1.ext4.cba /dev/mmcblk0p2
```

Clone and use output ( `/dev/mmcblk0p1` ) as seed while cloning:

```
upgrader@device:~$ bita clone --seed-output https://host/release_v1.1.ext4.cba /dev/mmcblk0p1
```

Local archives can also be cloned:

```
upgrader@device:~$ bita clone --seed-output local.cba local_output.file
```

Clone file at `https://host/new.tar.cba` using stdin (-) and block device `/dev/sda1` as seed:

```
olle@home:~$ gunzip -c old.tar.gz | bita clone --seed /dev/sda1 --seed - https://host/new.tar.cba new.ta
```

Compare two filesystem images to see how much content they share with different chunking parameters:

```
olle@home:~$ bita diff release_v1.0.ext4 release_v1.1.ext4
olle@home:~$ bita diff --hash-chunking BuzHash --avg-chunk-size 8KiB release_v1.0.ext4 release_v1.1.ext4
```

## Similar tools and inspiration

- casync
- zchunk
- zsync
- rsync
- bup

### Metadata

3 months ago
2021 edition
MIT
203 KiB

### Install

```
cargo install bita
```

Running the above command will globally install the **bita** binary.

### Repository

github.com/oll3/bita

### Owners


Olle Sandberg

### Categories

- Command line utilities
- Compression
- Filesystem

**Report crate**

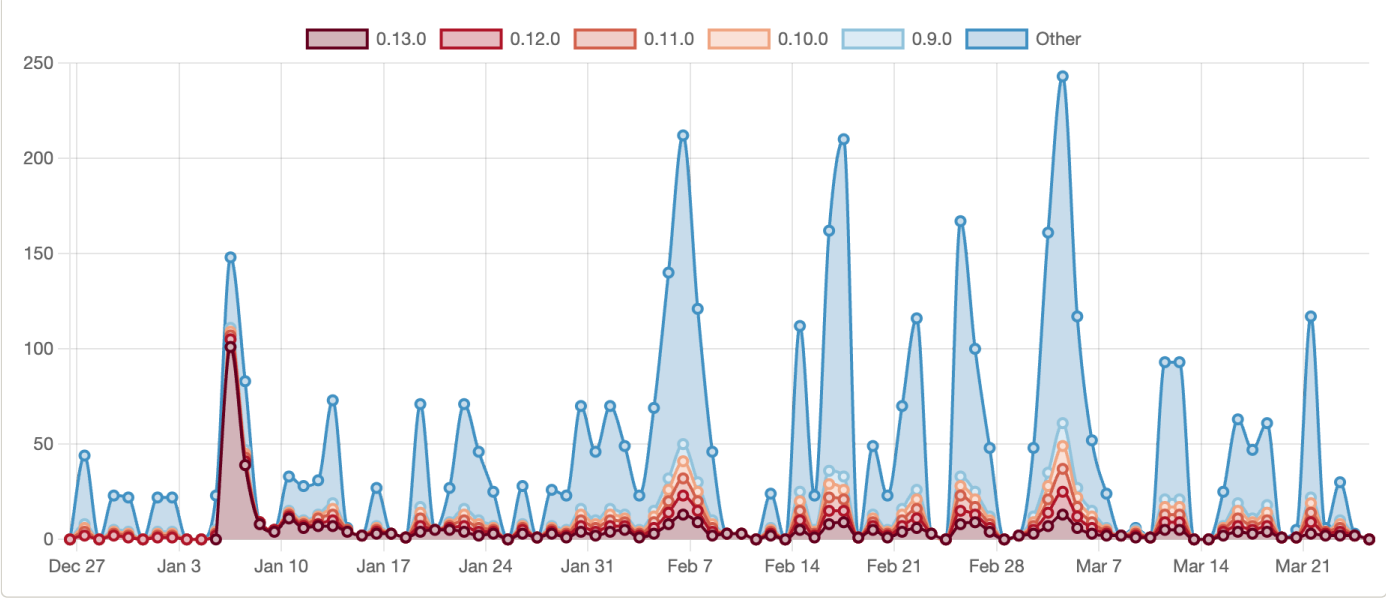
---

### Stats Overview

**23,062**
Downloads all time

**23**
Versions published

**Downloads over the last 90 days** Display as Stacked




---

## Rust

rust-lang.org

Rust Foundation

The crates.io team

## Get Help

The Cargo Book

Support

System Status

Report a bug

## Policies

Usage Policy

Security

Privacy Policy

Code of Conduct

Data Access

## Social

rust-lang/crates.io

#t-crates-io

@cratesiostatus